

gcongr

Pascal Herbrich

June 2026

```
example {a b : ℕ} : a + b ≤ b + a := by gcongr; sorry; sorry
```

Overview

What can gcongr do?

Overview

What can gcongr do?

The 'gongr' tactic applies 'generalized congruence' rules, reducing a relational goal between a LHS and RHS.

Overview

What can gcongr do?

The 'gongr' tactic applies 'generalized congruence' rules, reducing a relational goal between a LHS and RHS.

- Which rules are 'generalized congruence' rules?

Overview

What can gcongr do?

The 'gongr' tactic applies 'generalized congruence' rules, reducing a relational goal between a LHS and RHS.

- Which rules are 'generalized congruence' rules?
- How can we identify them?

Overview

What can gcongr do?

The 'gongr' tactic applies 'generalized congruence' rules, reducing a relational goal between a LHS and RHS.

- Which rules are 'generalized congruence' rules?
- How can we identify them?
- What is a relational goal?

Overview

What can gcongr do?

The 'gongr' tactic applies 'generalized congruence' rules, reducing a relational goal between a LHS and RHS.

- Which rules are 'generalized congruence' rules?
- How can we identify them?
- What is a relational goal?
- What should the LHS and RHS look like?

Examples (yeah!)

Overview

Options to use gcongr

Overview

Options to use `gcongr`

- `gcongr` with `x y ...z` or `gcongr` with $\langle a, b \rangle c$
- `gcongr` `n` with $n \in \mathbb{N}$
- `gcongr` `t` for a template `t`

Outline

- Relations
- Generalized congruence lemmas
- Forward reasoning
- Side goals
- Main function
- Rel tactic
- Examples

Relations

Relations

There are two types

Relations

There are two types

- takes lhs, rhs and returns a Prop

Relations

There are two types

- takes lhs, rhs and returns a Prop
- implications

Generalized congruence lemmas

Generalized congruence lemmas

- all lemmas tagged with `@[(kind) gcongr (prio)]`

Generalized congruence lemmas

- all lemmas tagged with `@[(kind) gcongr (prio)]`
- goal has the form

$$f\ a_1 \dots a_n \sim f\ b_1 \dots b_n$$

Generalized congruence lemmas

Special cases

Generalized congruence lemmas

Special cases

- iff lemmas tried to add one implication

Generalized congruence lemmas

Special cases

- iff lemmas tried to add one implication
- tries to add the goal as an implication from a hypothesis

Forward reasoning

- has it's own @[gcongr_forward]

Forward reasoning

- has it's own @[gcongr_forward]
- applies a hypothesis in some way on the goal

Forward reasoning

- has it's own @[gcongr_forward]
- applies a hypothesis in some way on the goal
- tries all forward tactics on all hypotheses and their symm form

Forward reasoning

Tried every time

- exactRefl
- exact

Side goals

Side goals

- runs own tactic `gcongr_discharger`

Side goals

- runs own tactic `gcongr_discharger`
- this executes `positivity` and `assumption`

Main function

Rel tactic

Rel tactic

- uses same function as gcongr

Rel tactic

- uses same function as `gcongr`
- finishing tactic (!)

Rel tactic

- uses same function as `gcongr`
- finishing tactic (!)
- only option to use is as `rel [h1 ... hn]` for hypotheses `hi`

Rel tactic

- uses same function as `gcongr`
- finishing tactic (!)
- only option to use is as `rel [h1 ... hn]` for hypotheses `hi`
- uses only those hypotheses for the forward reasoning

Examples

```
example {a : ℕ} : a ≥ 0 := by
  congr
```

```
example {a : ℕ} (h: a ≥ 0) : a ≥ 0 := by
  congr
```

```
example {a : ℕ} : a ≥ 0 := by
  congr_discharger
```

```
example {a : ℕ} (h: a ≥ 0) : a ≥ 0 := by
  congr_discharger
```

@[gcongr]

theorem of_eq_r {a b : $\mathbb{N}$ } (h : a = b) [Std.Refl r] : r a b := **by**
 exact of_eq h -- *this solves the goal*

example {a b c : $\mathbb{N}$ } (h : a = b) [Std.Refl r]: r (a + c) (b +
 c) := **by**
 gcongr
 linarith

```

@[gcongr]
theorem of_eq_r_2 {a b : ℕ} (h : a = b) [Std.Refl r] : r (id a)
  (id b) := by
  exact of_eq h -- this solves the goal

example {a b c : ℕ} (h : a = b) [Std.Refl r]: r (id (a + c))
  (id (b + c)) := by
  gcongr
  linarith

```

```
@[gcongr only]
lemma my_imp {a b : ℕ} : r a b → r b a := by
  sorry

example {a b : ℕ} : r b a → r a b := by
  gcongr
```

```

example (h : a → b) : (a ∧ ¬b) ∨ c → (b ∧ ¬a) ∨ c := by
  congr
example (h : a → b) : (a ∧ ¬b) ∨ c → (b ∧ ¬a) ∨ c := by
  congr ?_
example (h : a → b) : (a ∧ ¬b) ∨ c → (b ∧ ¬a) ∨ c := by
  congr ?_ ∧ ¬?_ ∨ c

```



```
example {x y :  $\mathbb{Z}$ } (hx :  $x \geq 12$ ) :  $y + x * x \geq y + 12 * x$  := by
  rel [hx]
```